StepUp - A Step Goal Early-Warning Predictor - Modeling

1. Problem Statement

Problem: I have a step goal of 10k per day. After a long afternoon, I've gotten halfway through that goal. But when I reach home, all I want to do is chill on my couch or just take a nap, then things get busy and I end up staying halfway through that goal for the rest of the day.

Objective: What this project aims to do is to predict, based on my current steps taken and time of day, whether I'll be able to reach that goal or not. It'll also say how much extra walking would improve my chances of reaching the goal in time.

Why it matters: When I get home I tend to skip a walk, and the shortfall only shows up when it's too late to fix. An early warning turns that into something I can still act on.

Expected outcome: A trained classifier, an evaluation, and a working frontend that serves a clear purpose. My classifier should beat a simple baseline.

2. Loading & Filtering Data

2.1 Load Processed Files

In [1]:
```
import pandas as pd
```

In [2]:
```
minute = pd.read_csv('./data/processed_minute_steps.csv', parse_dates=['ActivityMin
daily = pd.read_csv('./data/processed_daily.csv', parse_dates=['ActivityDate'], dat
# Both files come from 01_eda_cleaning.ipynb
```

In [3]:
```
minute.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1325580 entries, 0 to 1325579
Data columns (total 5 columns):
 #   Column          Non-Null Count    Dtype         
---  ------          --------------    -----         
 0   Id              1325580 non-null  int64         
 1   ActivityMinute  1325580 non-null  datetime64[ns]
 2   Steps           1325580 non-null  int64         
 3   ActivityDate    1325580 non-null  object        
 4   Hour            1325580 non-null  int64         
dtypes: datetime64[ns](1), int64(3), object(1)
memory usage: 50.6+ MB
```

In [4]:
```
daily.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 934 entries, 0 to 933
Data columns (total 17 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   Id                         934 non-null    int64
 1   ActivityDate               934 non-null    datetime64[ns]
 2   Steps                      934 non-null    int64
 3   TotalSteps                 934 non-null    int64
 4   TotalDistance              934 non-null    float64
 5   TrackerDistance            934 non-null    float64
 6   LoggedActivitiesDistance   934 non-null    float64
 7   VeryActiveDistance         934 non-null    float64
 8   ModeratelyActiveDistance   934 non-null    float64
 9   LightActiveDistance        934 non-null    float64
 10  SedentaryActiveDistance    934 non-null    float64
 11  VeryActiveMinutes          934 non-null    int64
 12  FairlyActiveMinutes        934 non-null    int64
 13  LightlyActiveMinutes       934 non-null    int64
 14  SedentaryMinutes           934 non-null    int64
 15  Calories                   934 non-null    int64
 16  StepsDifference            934 non-null    int64
dtypes: datetime64[ns](1), float64(7), int64(9)
memory usage: 124.2 KB
```

2.2 Remove High-Mismatch Users

Before I train my model on data that's present in two different files, I need to make sure that the data I'm using is consistent across files. So I'll need to determine which users have a high disagreement between their daily and minute-level step counts, and drop them before I train.

I'll take some code from 01_eda_cleaning, with some changes to include users with zero mismatches instead of just showing NaN:

In [5]:
```
mismatches = daily[daily['StepsDifference'] != 0] # Only the rows with a complete a
```

In [6]:
```
total_days = daily.groupby('Id').size() # The total number of days for each Id
mismatch_counts = mismatches['Id'].value_counts() # The number of rows that disagre

# Calculate the percentage of disagreement:
mismatch_rate = round(((mismatch_counts.reindex(total_days.index, fill_value=0) / t
mismatch_rate
```

Out[6]:
```
Id
8877689391    83.9
8583815059    50.0
4020332650    48.4
7086361926    35.5
4319703577    29.0
4057192912    25.0
1503960366    23.3
2320127002    22.6
1644430081    20.0
3977333714    17.2
1624580081    16.1
4702921684    16.1
4445114986    16.1
5577150313    13.3
4558609924    12.9
8053475328    12.9
5553957443    12.9
6775888955    11.5
6117666160    10.7
2026352035     9.7
4388161847     9.7
8378563200     9.7
1844505072     6.5
2873212765     6.5
2347167796     5.6
8253242879     5.6
3372868164     5.0
6290855005     3.6
8792009665     3.6
6962181067     3.2
2022484408     3.2
1927972279     0.0
7007744171     0.0
dtype: float64
```

I will drop users whose minute-level and daily steps disagreed on more than 40% of their days, which is where the data shows a clear separation. There's no clear break below the top 3, and dropping more users costs me data I don't have much of, so 40% should be a reasonable threshold.

In [7]:
```
drop_list = mismatch_rate[mismatch_rate > 40].index
drop_list
```

Out[7]:
```
Index([8877689391, 8583815059, 4020332650], dtype='int64', name='Id')
```

In [8]:
```
keep_daily = ~daily['Id'].isin(drop_list)
daily_cleaned = daily[keep_daily].copy()
```

In [9]:
```
keep_minute = ~minute['Id'].isin(drop_list)
minute_cleaned = minute[keep_minute].copy()
```

In [10]:
```
daily_cleaned.shape
```

Out[10]: (842, 17)

In [11]:
```
minute_cleaned.shape
```

Out[11]: (1194540, 5)

This process removed 3 of 33 users, leaving 30.

Daily rows: 934 -> 842 (92 rows dropped)

Minute rows: 1,325,580 -> 1,194,540 *(131,040 rows dropped)*

2.3 Keep Complete Days Only

Since the model predicts whether you'll reach the goal by the end of the day or not, it should only train on complete days, or else it could give a false prediction like "you'll reach the goal by today" if all it has to base that prediction off is an incomplete day.

In [12]:
```
minute_cleaned['ActivityDate'] = minute_cleaned['ActivityMinute'].dt.normalize()
```

In [13]:
```
minute_cleaned.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1194540 entries, 0 to 1281539
Data columns (total 5 columns):
 #   Column          Non-Null Count    Dtype         
---  ------          --------------    -----         
 0   Id              1194540 non-null  int64         
 1   ActivityMinute  1194540 non-null  datetime64[ns]
 2   Steps           1194540 non-null  int64         
 3   ActivityDate    1194540 non-null  datetime64[ns]
 4   Hour            1194540 non-null  int64         
dtypes: datetime64[ns](2), int64(3)
memory usage: 54.7 MB
```

In [14]:
```
# This will be useful during model training. More on this later
# minute_cleaned['Hour'] = minute_cleaned['ActivityMinute'].dt.hour
```

In [15]:
```
minutes_per_day = minute_cleaned.groupby(['Id', 'ActivityDate']).size()
minutes_per_day # Gets the total amount of minutes per day for each user
```

Out[15]:
```
Id          ActivityDate
1503960366  2016-04-12      1440
            2016-04-13      1440
            2016-04-14      1440
            2016-04-15      1440
            2016-04-16      1440
                            ...
8792009665  2016-05-05      1440
            2016-05-06      1440
            2016-05-07      1440
            2016-05-08      1440
            2016-05-09      1440
Length: 842, dtype: int64
```

In [16]:
```
minutes_per_day.value_counts() # A "bird's eye" view of how many rows are actually
```

Out[16]:
```
1440    814
960       7
720       4
900       4
780       3
660       2
600       2
1260      1
360       1
240       1
1020      1
60        1
1380      1
Name: count, dtype: int64
```

814 user-days out of 842 are complete, with 28 days being the partial ones. I'll only be losing ~3% of data when I drop them, which isn't a lot.

In [17]:
```
# Since .size() only returns a collapsed version of my data, I need to use .transfo
# so I have a mask that's exactly 1,194,540 rows long, which can be used as a mask.
total_minutes_per_day = minute_cleaned.groupby(['Id', 'ActivityDate'])['ActivityDat
total_minutes_per_day.value_counts()
```

Out[17]:
```
ActivityDate
1440    1172160
960        6720
900        3600
720        2880
780        2340
1380       1380
660        1320
1260       1260
600        1200
1020       1020
360         360
240         240
60           60
Name: count, dtype: int64
```

In [18]:
```
minute_cleaned = minute_cleaned[total_minutes_per_day == 1440]
minute_cleaned.groupby(['Id', 'ActivityDate']).size().value_counts()
```

Out[18]:
```
1440    814
Name: count, dtype: int64
```

Doing it for the daily dataset too, so both tables describe the same user-days

In [19]:
```
complete_days = minute_cleaned[['Id', 'ActivityDate']].drop_duplicates()
complete_days.shape
```

Out[19]:
```
(814, 2)
```

In [20]:
```
daily_cleaned = pd.merge(daily_cleaned, complete_days, on=['Id', 'ActivityDate'])
```

In [21]:
```
len(daily_cleaned)
```

Out[21]:
```
814
```

In [22]:
```
daily_cleaned['Id'].nunique()
```

Out[22]:
```
30
```

In [23]:
```
len(minute_cleaned)
```

Out[23]:
```
1172160
```

3. Defining the Prediction Problem

3.1 Structure of the Training Table

Here's what the structure of the training table would look like:

Id	ActivityDate	MinuteOfDay	StepsSoFar	FinalTotal	Reached10k
A	Apr 20	600	1,200	11,400	1
A	Apr 20	870	3,000	11,400	1
A	Apr 21	1,110	700	4,300	0

*Illustrative values (600 = 10:00am, 870 = 2:30pm, 1,110 = 6:30pm)

We'll call this table `checkpoints`. The `checkpoints` table will be built from the minute-level step data. Each row contains a user, a day, and a time of day measured in minutes since midnight. Then we have the amount of steps taken so far, and our target column, which tells us whether that user reached 10k steps or not by the end of the day.

Each day produces one row per checkpoint, every 30 minutes from 8:00am to 9:30pm, so the model sees what “steps so far” looks like at different times of day, and the app needs that

because the user can ask at any time.

To prevent leakage: “steps so far” at a time like 3pm must count only minutes *before* 3pm (as far as an actual person could see at that point). If any later minute leaks in, that means the model is peeking at the answer, and it will score very well on the test set but be useless in an actual prediction because it's not actually predicting anything. The checkpoint minute itself shouldn’t be counted (at 3:00pm it'll count up to 2:59pm).

Moreover, `FinalTotal` and `Reached10k` will never be used as features.

3.2 Target Variable

The target column is `Reached10k`: 1 if the user’s full-day step total was 10,000 or more, otherwise it's 0. The full-day total is the sum of that day’s minute-level steps (the same source as the features), and only complete days are used (done in 2.3: 814 kept, 28 dropped), so a partial day can’t produce a false 0. I've picked 10k steps as a fixed goal as that's what the project is built around.

`FinalTotal` is stored in the table next to the target.

All of a day’s checkpoints share the same value, since the outcome belongs to the day, not the checkpoint.

4. Feature Engineering

In [24]:
```
minute_cleaned = minute_cleaned.sort_values(by=['Id', 'ActivityMinute'])
```

In [25]:
```
minute_cleaned['MinuteOfDay'] = minute_cleaned['ActivityMinute'].dt.hour * 60 + min
```

In [26]:
```
# This gets the total of steps per user-day, but then subtracting it with the steps
# in the current minute, which gives us our "steps so far" - steps taken strictly b
minute_cleaned['StepsSoFar'] = minute_cleaned.groupby(['Id', 'ActivityDate'])['Step
```

In [27]:
```
minute_cleaned['FinalTotal'] = minute_cleaned.groupby(['Id', 'ActivityDate'])['Step
```

In [28]:
```
minute_cleaned['Reached10k'] = (minute_cleaned['FinalTotal'] >= 10000).astype(int)
```

In [29]:
```
checkpoints = minute_cleaned[(minute_cleaned['MinuteOfDay'] % 30 == 0) & (minute_cl
```

In [30]:
```
checkpoints = checkpoints[checkpoints['FinalTotal'] > 0] # not keeping rows where n
```

In [31]:
```
checkpoints.drop(['ActivityMinute', 'Steps', 'Hour'], axis=1, inplace=True)
```

In [32]:
```
checkpoints = checkpoints.reset_index(drop=True)
checkpoints
```

Out[32]:

	Id	ActivityDate	MinuteOfDay	StepsSoFar	FinalTotal	Reached10k
0	1503960366	2016-04-12	480	684	13158	1
1	1503960366	2016-04-12	510	684	13158	1
2	1503960366	2016-04-12	540	934	13158	1
3	1503960366	2016-04-12	570	1922	13158	1
4	1503960366	2016-04-12	600	2798	13158	1
...	...	...	...	...	...	...
21191	8792009665	2016-05-04	1170	2151	2283	0
21192	8792009665	2016-05-04	1200	2151	2283	0
21193	8792009665	2016-05-04	1230	2255	2283	0
21194	8792009665	2016-05-04	1260	2283	2283	0
21195	8792009665	2016-05-04	1290	2283	2283	0

21196 rows × 6 columns

In [33]:
```
checkpoints.isna().sum()
```

Out[33]:
```
Id              0
ActivityDate    0
MinuteOfDay     0
StepsSoFar      0
FinalTotal      0
Reached10k      0
dtype: int64
```

5. Train/Test Split & Setting a Baseline

5.1 Splitting the data

I won't be using sklearn's train_test_split, because it defaults to shuffling rows by random, so the checkpoints of one day could land in both the train and test set. The model would then be tested on days it has already seen during training and the score will be inflated.

As a workaround, I'll choose a cutoff date instead. I'll train on the days before it, and test on the days from it onward. That also matches how the model will actually be used, since it predicts days it hasn't seen.

In [34]:
```
sorted(checkpoints['ActivityDate'].unique())
```

Out[34]:
```
[Timestamp('2016-04-12 00:00:00'),
 Timestamp('2016-04-13 00:00:00'),
 Timestamp('2016-04-14 00:00:00'),
 Timestamp('2016-04-15 00:00:00'),
 Timestamp('2016-04-16 00:00:00'),
 Timestamp('2016-04-17 00:00:00'),
 Timestamp('2016-04-18 00:00:00'),
 Timestamp('2016-04-19 00:00:00'),
 Timestamp('2016-04-20 00:00:00'),
 Timestamp('2016-04-21 00:00:00'),
 Timestamp('2016-04-22 00:00:00'),
 Timestamp('2016-04-23 00:00:00'),
 Timestamp('2016-04-24 00:00:00'),
 Timestamp('2016-04-25 00:00:00'),
 Timestamp('2016-04-26 00:00:00'),
 Timestamp('2016-04-27 00:00:00'),
 Timestamp('2016-04-28 00:00:00'),
 Timestamp('2016-04-29 00:00:00'),
 Timestamp('2016-04-30 00:00:00'),
 Timestamp('2016-05-01 00:00:00'),
 Timestamp('2016-05-02 00:00:00'),
 Timestamp('2016-05-03 00:00:00'),
 Timestamp('2016-05-04 00:00:00'),
 Timestamp('2016-05-05 00:00:00'),
 Timestamp('2016-05-06 00:00:00'),
 Timestamp('2016-05-07 00:00:00'),
 Timestamp('2016-05-08 00:00:00'),
 Timestamp('2016-05-09 00:00:00'),
 Timestamp('2016-05-10 00:00:00'),
 Timestamp('2016-05-11 00:00:00')]
```

In [35]:
```
print(checkpoints['ActivityDate'].nunique())
```

```
30
```

With 30 dates, a test set of 20% is the last 6 days. So I'll make the cut at May 6 to 11 for the test, and the prior will be for train (April 12 to May 5).

In [36]:
```
is_train = checkpoints['ActivityDate'] < '2016-05-06'
train_df = checkpoints[is_train].copy()
```

In [37]:
```
is_test = ~is_train # flipped
test_df = checkpoints[is_test].copy()
```

In [38]:
```
sorted(train_df['ActivityDate'].unique())
```

Out[38]:
```
[Timestamp('2016-04-12 00:00:00'),
 Timestamp('2016-04-13 00:00:00'),
 Timestamp('2016-04-14 00:00:00'),
 Timestamp('2016-04-15 00:00:00'),
 Timestamp('2016-04-16 00:00:00'),
 Timestamp('2016-04-17 00:00:00'),
 Timestamp('2016-04-18 00:00:00'),
 Timestamp('2016-04-19 00:00:00'),
 Timestamp('2016-04-20 00:00:00'),
 Timestamp('2016-04-21 00:00:00'),
 Timestamp('2016-04-22 00:00:00'),
 Timestamp('2016-04-23 00:00:00'),
 Timestamp('2016-04-24 00:00:00'),
 Timestamp('2016-04-25 00:00:00'),
 Timestamp('2016-04-26 00:00:00'),
 Timestamp('2016-04-27 00:00:00'),
 Timestamp('2016-04-28 00:00:00'),
 Timestamp('2016-04-29 00:00:00'),
 Timestamp('2016-04-30 00:00:00'),
 Timestamp('2016-05-01 00:00:00'),
 Timestamp('2016-05-02 00:00:00'),
 Timestamp('2016-05-03 00:00:00'),
 Timestamp('2016-05-04 00:00:00'),
 Timestamp('2016-05-05 00:00:00')]
```

In [39]:
```
train_df['ActivityDate'].nunique()
```

Out[39]:
```
24
```

In [40]:
```
sorted(test_df['ActivityDate'].unique())
```

Out[40]:
```
[Timestamp('2016-05-06 00:00:00'),
 Timestamp('2016-05-07 00:00:00'),
 Timestamp('2016-05-08 00:00:00'),
 Timestamp('2016-05-09 00:00:00'),
 Timestamp('2016-05-10 00:00:00'),
 Timestamp('2016-05-11 00:00:00')]
```

In [41]:
```
test_df['ActivityDate'].nunique()
```

Out[41]:
```
6
```

Now I'll proceed to manually splitting train & test:

In [42]:
```
X_train = train_df[['MinuteOfDay', 'StepsSoFar']]
X_test = test_df[['MinuteOfDay', 'StepsSoFar']]
y_train = train_df['Reached10k']
y_test = test_df['Reached10k']
```

In [43]:
```
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
print(y_train.mean(), y_test.mean())
```

```
(17864, 2) (3332, 2) (17864,) (3332,)
0.3448275862068966 0.36134453781512604
```

The split seems good, the shapes line up (17,864 + 3,332 = 21,196)

Test size is about 15.7%, which is a bit under 20% because the last few dates have fewer complete user-days, but is not really a big gap to need a re-split, so I'll just keep it as it is.

Class balance is almost identical with 34.4% in train and 36.1% in test, so the test set represents the same mix of outcomes.

5.2 Scoring the baseline

Before training the models, I need baselines for them to beat.

I'll use two. The first is an always-predict-0 baseline, using sklearn's DummyClassifier, which shows why accuracy alone is misleading for this problem. Technically its accuracy could also be computed directly as `1 - y_test.mean()`, but the dummy classifier lets me score it with the same metrics as the real models (precision, recall, F1 and confusion matrix), so every comparison uses the same measurements.

The second baseline is a pace-projection rule, which we'll get into once we're done with the DummyClassifier prediction below.

In [44]:
```
from sklearn.dummy import DummyClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

In [45]:
```
dummy_clf = DummyClassifier(strategy="most_frequent") # always predict whichever cl
```

In [46]:
```
dummy_clf.fit(X_train, y_train) # ignores X_train completely and just remembers the
```

Out[46]:

▾ DummyClassifier

▶ Parameters

In [47]:
```
dummy_pred = dummy_clf.predict(X_test)
```

In [48]:
```
# Code from Gemini - I already know what this is and what it does, just needed to a
def score_model(y_true, y_pred, title):
    """
    Prints standard classification metrics and a confusion matrix.

    Parameters:
    y_true (array-like): Ground truth (correct) target values.
    y_pred (array-like): Estimated targets as returned by a classifier.
    title (str): The name or strategy of the classifier for the print header.
    """
    print(f"--- {title} Performance Metrics ---")

    # zero_division=0 just prevents a warning from cluttering the output
    print(f"Accuracy Score : {accuracy_score(y_true, y_pred):.4f}")
    print(f"Precision Score: {precision_score(y_true, y_pred, zero_division=0):.4f}
```

```
    print(f"Recall Score   : {recall_score(y_true, y_pred, zero_division=0):.4f}")
    print(f"F1 Score       : {f1_score(y_true, y_pred, zero_division=0):.4f}")

    print("\n--- Confusion Matrix ---")
    print(confusion_matrix(y_true, y_pred))
```

In [49]:
```
score_model(y_test, dummy_pred, 'Dummy Classifier')
```

```
--- Dummy Classifier Performance Metrics ---
Accuracy Score : 0.6387
Precision Score: 0.0000
Recall Score   : 0.0000
F1 Score       : 0.0000

--- Confusion Matrix ---
[[2128    0]
 [1204    0]]
```

As we can see above, its accuracy is 63.8%, but recall and F1 are 0 because it never predicts that a user will reach 10k, so it would never give anyone a useful warning. This shows that accuracy alone is a poor measure for this problem, and that a model has to do better on precision, recall and F1 as well.

Now we'll do the pace-prediction rule, which is a fair test of whether machine learning adds anything. It asks whether the person is on pace for 10k if they keep the typical shape of a day.

In [50]:
```
train_df['Progress'] = train_df['StepsSoFar'] / train_df['FinalTotal']
```

In [51]:
```
train_df.sample(5)
```

Out[51]:

	Id	ActivityDate	MinuteOfDay	StepsSoFar	FinalTotal	Reached10k	Progr
3534	2022484408	2016-04-12	660	6667	11875	1	0.5614
1760	1644430081	2016-04-15	1200	3580	5263	0	0.6802
16780	6962181067	2016-05-03	720	5263	12109	1	0.4346
17949	7086361926	2016-04-22	510	964	9733	0	0.0990
15330	6290855005	2016-04-20	900	5170	6361	0	0.8127

In [52]:
```
progress_curve = train_df.groupby('MinuteOfDay')['Progress'].mean()
```

In [53]:
```
progress_curve
```

Out[53]:
```
MinuteOfDay
480     0.100787
510     0.126341
540     0.158084
570     0.183857
600     0.212064
630     0.243099
660     0.275572
690     0.306377
720     0.337305
750     0.372330
780     0.403732
810     0.436208
840     0.470481
870     0.506149
900     0.534734
930     0.561314
960     0.586006
990     0.614501
1020    0.645408
1050    0.677585
1080    0.713784
1110    0.751206
1140    0.789734
1170    0.828597
1200    0.861195
1230    0.886494
1260    0.909032
1290    0.929535
Name: Progress, dtype: float64
```

For each training row, I compute Progress `(StepsSoFar / FinalTotal)`, the share of the day's steps already taken at that checkpoint, and average it for each time of day to get `progress_curve`, the typical shape of a day.

In [54]:
```
typical_progress = X_test['MinuteOfDay'].map(progress_curve)
typical_progress.isna().sum()
```

Out[54]:
```
np.int64(0)
```

In [55]:
```
projected_total = test_df['StepsSoFar'] / typical_progress
```

In [56]:
```
projected_total
```

Out[56]:
```
672       3175.003870
673       2611.987053
674       2947.804201
675       7527.603074
676      10133.738840
             ...
20659    10439.330419
20660    10063.925319
20661    10060.987218
20662     9811.537857
20663     9737.126985
Length: 3332, dtype: float64
```

In [57]:
```
pace_pred = (projected_total >= 10000).astype(int)
```

In [58]:
```
score_model(y_test, pace_pred, 'Pace Prediction')
```

```
--- Pace Prediction Performance Metrics ---
Accuracy Score : 0.7239
Precision Score: 0.6464
Recall Score   : 0.5208
F1 Score       : 0.5768

--- Confusion Matrix ---
[[1785  343]
 [ 577  627]]
```

To predict, I divide a row's steps so far by the curve value for its time to get a projected daily total, and predict 1 if that reaches 10,000. E.g. if people are typically 50% done by 2pm and someone has 4,000 steps, the projection is 8,000, so the prediction is 0.

The curve is built from training days only, and a test row only supplies its time and steps so far, so there's no leakage. This is the rule a person would naturally use, so the models need to beat it to show they add value.

Here's what this pace baseline tells me:

- Accuracy is 0.724 against 0.638 for DummyClassifier, so the rule is clearly better, and it actually warns, with recall of 0.52.
- Of the 970 checkpoints it predicted would reach 10k, 64.6% did. Of the 1,204 checkpoints that really reached 10k, it caught 627 (52%) and missed 577.
- The two kinds of mistake are different. 343 were "you can reach it" when the person then missed the goal, and 577 were "you won't make it" when the person actually would have.

The bar for the models: accuracy above 0.724 and F1 above 0.577, and a better confusion matrix.

6. Modeling

6.1 Selected Models and Why

I've chosen Logistic Regression and Random Forest as the models I'd want to compare.

Why?

- **Logistic Regression:** It's simple and interpretable, and it gives a probability, which fits the "62% likely to reach your goal" output. With these two features it can only draw a straight-line boundary between "will reach" and "won't".
- **Random Forest:** It can learn that what counts as "on pace" changes with the time of day, which a straight line can't do. That's why the comparison will be interesting (linear vs flexible)

6.2 Model 1 - Logistic Regression

Logistic Regression is sensitive to feature scales, so I'll have to scale the features with StandardScaler before training on them.

In [59]:
```
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
```

In [60]:
```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

In [61]:
```
lr = LogisticRegression(max_iter=1000, random_state=42)
lr.fit(X_train_scaled, y_train)
```

Out[61]:

▾ LogisticRegression

▶ Parameters

In [62]:
```
lr_pred = lr.predict(X_test_scaled)
lr_proba = lr.predict_proba(X_test_scaled)[:, 1] # to get only the probability of r
```

In [63]:
```
score_model(y_test, lr_pred, "Logistic Regression")
```

```
--- Logistic Regression Performance Metrics ---
Accuracy Score : 0.7509
Precision Score: 0.7373
Recall Score   : 0.4826
F1 Score       : 0.5833

--- Confusion Matrix ---
[[1921  207]
 [ 623  581]]
```

What I can see from this:

- The model beats the pace baseline on accuracy and precision, and ties on F1 score, so the overall gain is modest.
- It's more careful about the costly mistake: it says "you can reach it" wrongly 207 times instead of 343. It catches 90.3% of real shortfalls (1,921 of 2,128) against 83.9% for the pace rule.
- It pays for that with slightly lower recall. It misses more days that would have reached 10k, but the share of its "won't make it" warnings that are unnecessary is almost the same as the baseline's.
- A modest gain makes sense, I have only two features, and the pace rule is already a strong baseline.

6.3 Model 2 - Random Forest

In [64]:
```
from sklearn.ensemble import RandomForestClassifier
```

In [65]:
```
rf = RandomForestClassifier(max_depth=8, min_samples_leaf=30, n_estimators=200, ran
rf.fit(X_train, y_train)
```

Out[65]:

▾ RandomForestClassifier

▶ Parameters

In [66]:
```
rf_pred = rf.predict(X_test)
rf_proba = rf.predict_proba(X_test)[:, 1]
```

In [67]:
```
score_model(y_test, rf_pred, "Random Forest")
```

```
--- Random Forest Performance Metrics ---
Accuracy Score : 0.7683
Precision Score: 0.8927
Recall Score   : 0.4078
F1 Score       : 0.5599

--- Confusion Matrix ---
[[2069   59]
 [ 713  491]]
```

Comparison table between the pace rule baseline, and the models Logistic Regression, and Random Forest.

	Pace rule	Logistic Regression	Random Forest
Accuracy	0.724	0.751	**0.768**
Precision	0.646	0.737	**0.893**
Recall	0.521	**0.483**	0.408
F1	0.577	**0.583**	0.560

	Pace rule	Logistic Regression	Random Forest
Wrong "you will make it"	343	207	**59**
Unnecessary warnings	577	623	713
Real shortfalls caught	83.9%	90.3%	**97.2%**

- Random Forest is the most "safe". When it says you'll reach 10k, it's right 89% of the time, and it catches 97% of real shortfalls (2,069 of 2,128).
- The price is that it misses more days that would have reached 10k (713). Even so, the share of its unnecessary warnings (about 25.6%) is close to Logistic Regression's (about 24.5%).
- F1 is lower than Logistic Regression's because F1 rewards recall. Which model is better depends on which mistake matters more, and for a warning app that's the wrong "you will make it", where Random Forest is clearly best.

Random Forest seems like the choice to make here, but both models are judged at the same 0.5 cutoff, which hides how well they rank days. So before I make a final decision, I'll do a quick ROC AUC check on both the models so I can see if Random Forest is actually better or just more cautious at 0.5.

In [68]:
```
from sklearn.metrics import roc_auc_score
```

In [69]:
```
roc_auc_score(y_test, lr_proba) # Logistic Regression
```

Out[69]:
```
0.742843209040042
```

In [70]:
```
roc_auc_score(y_test, rf_proba) # Random Forest
```

Out[70]:
```
0.7754922501436314
```

Random Forest scores 0.775 against 0.743 for Logistic Regression, so it ranks days better and isn't just more cautious at the 0.5 cutoff.

AUC of 0.5 is a coin flip and 1.0 is perfect, so 0.775 is decent but not great, which is reasonable because it only has two features to train on.

For the app, I'll use Random Forest. Since the app exists to warn people early, the costly mistake is saying "you will make it" to someone who then misses the goal. Random Forest made that mistake only 59 times on the test set (Logistic Regression: 207, pace rule: 343) and caught 97.2% of real shortfalls.

The trade-off is a lower recall and F1 than Logistic Regression. It gives more unnecessary warnings (713), which only cost the user a short walk. Logistic Regression would give smoother probabilities, but it catches fewer shortfalls.

7. Evaluation & Interpretation

7.1 Time-of-day Breakdown

In [71]:
```
# Code from Claude
res = test_df[['MinuteOfDay', 'Reached10k']].copy()
res['Dummy'], res['Pace'], res['LR'], res['RF'] = dummy_pred, pace_pred, lr_pred, r
res['TimeOfDay'] = pd.cut(res['MinuteOfDay'], bins=[479, 719, 1019, 1290],
                          labels=['Morning', 'Afternoon', 'Evening']) # 8am-11:59,

models = ['Dummy', 'Pace', 'LR', 'RF']

# a prediction is "correct" when it equals the true label, the mean of True/False i
correct = res[models].eq(res['Reached10k'], axis=0)
print(correct.groupby(res['TimeOfDay'], observed=True).mean().round(3))
print(res['TimeOfDay'].value_counts()) # rows per block
```

```
           Dummy   Pace     LR     RF
TimeOfDay
Morning    0.639  0.589  0.651  0.673
Afternoon  0.639  0.697  0.713  0.743
Evening    0.639  0.859  0.868  0.870
TimeOfDay
Afternoon    1190
Evening      1190
Morning       952
Name: count, dtype: int64
```

I divided the test set into three time blocks (Morning: 952 rows, Afternoon: 1,190 rows, Evening: 1,190 rows).

It seems that the model accuracy converges as the day goes on. In the morning, none of the models do much better than the dummy (LR 0.651 and RF 0.673 vs 0.639), so early-morning predictions are close to guessing. In fact, the simple pace rule actually performs worse than the dummy baseline in the morning (0.589 vs 0.639).

Likely why: Early-day steps are small and noisy, while late-day steps account for nearly the entire daily total.

App takeaway: The afternoon is the most practical window for predictions before the outcome is locked in by evening.

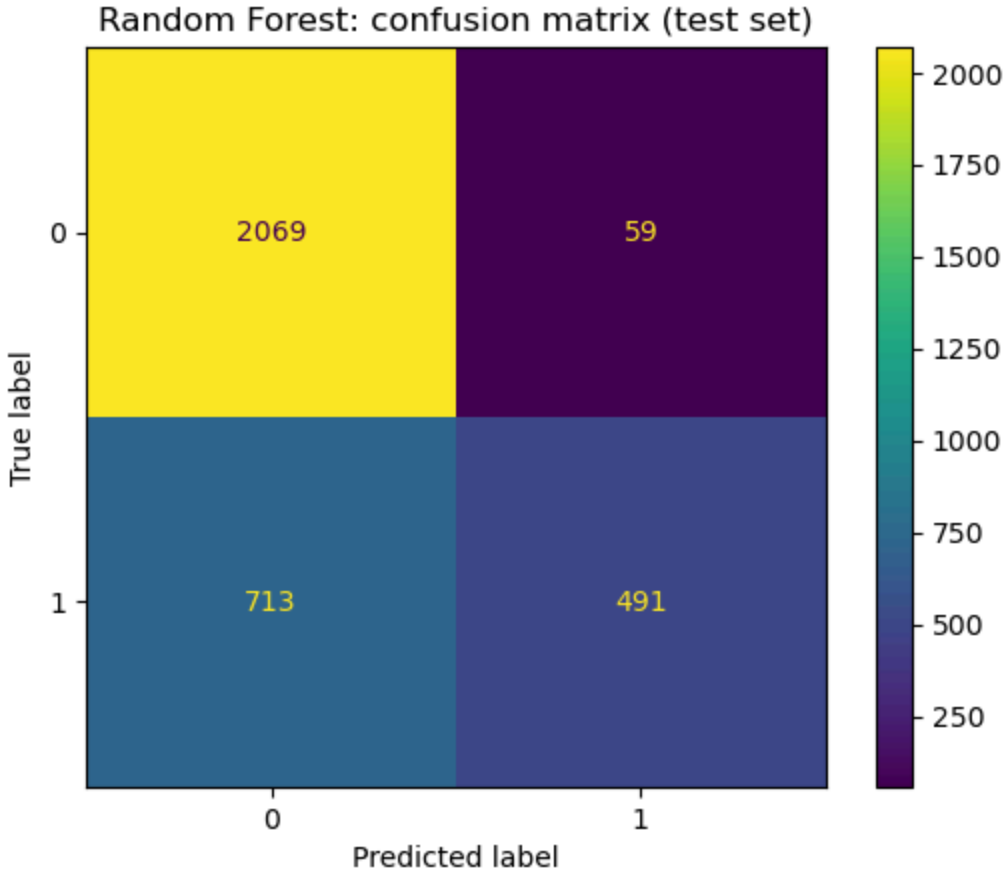
A caveat to consider is that this is based on a relatively small test set evaluated on accuracy alone.

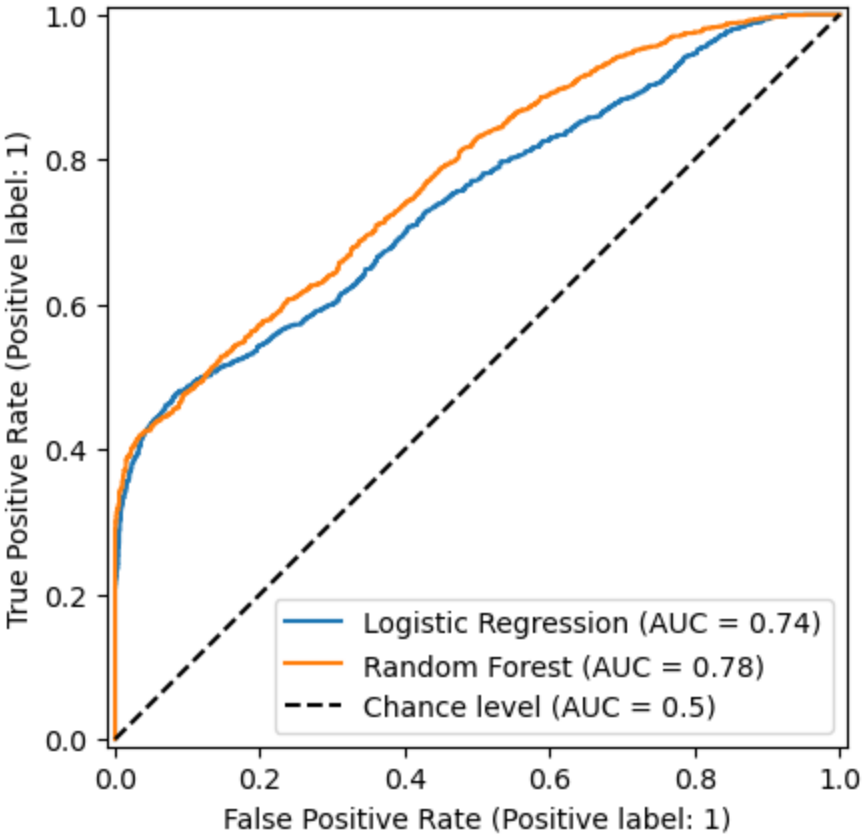
7.2 Plots

In [72]:
```
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay, RocCurveDisplay
```

```
ConfusionMatrixDisplay.from_predictions(y_test, rf_pred)
plt.title('Random Forest: confusion matrix (test set)')
plt.show()

fig, ax = plt.subplots()
RocCurveDisplay.from_predictions(y_test, lr_proba, name='Logistic Regression', ax=a
RocCurveDisplay.from_predictions(y_test, rf_proba, name='Random Forest', ax=ax, plo
plt.show()
```





Confusion matrix (Random Forest)

- Rows are the actual outcome and columns are the prediction. The top-left (2,069) are shortfalls correctly flagged, and the top-right (59) is the costly mistake of saying "you'll make it" when the user doesn't.
- The bottom-left (713) are the unnecessary warnings when the user would have made it anyway. That's what lowers recall.
- Together this shows the model is built to be cautious: 97% of shortfalls are caught, but it only recognises about 41% of the days that really reach 10k (491 of 1,204). For a warning app that's an acceptable trade, since the cost is just an extra walk.

ROC curve

- Both curves are well above the diagonal chance line, so both models carry real signal. RF is higher overall (AUC 0.78 vs 0.74).
- At low false-positive rates the two curves overlap, so RF's advantage appears only when I accept more false alarms. With a small test set, the gap is modest.
- Because the curve flattens, lowering the cutoff below 0.5 would catch more successful days at the cost of more false alarms.

8. [X] Calorie Regression (Secondary Model)

This section was meant to be a secondary model that would estimate total daily calories burned from a day's total steps (using Linear Regression), so the app could show what extra steps meant in energy terms. But I dropped it to focus on the main models and demo. It is listed under **11. Future Improvements**.

9. Save Trained Model

In [73]:
```
import os, joblib

os.makedirs("models", exist_ok=True)
joblib.dump(rf, "models/rf_model.joblib")
progress_curve.to_json("models/progress_curve.json")
```

10. Conclusion & Limitations

Findings: On the test set, the Random Forest reached 0.768 accuracy, 0.893 precision and 0.775 ROC AUC. It said "you'll make it" wrongly only 59 times (Logistic Regression: 207, pace rule: 343) and caught 97.2% of real shortfalls. Accuracy differences between the models shrink through the day, and in the morning none of the models do much better than always predicting 0.

Was the objective met? Yes, partly. The Random Forest beat the pace-rule baseline on accuracy, precision, false positives and AUC, but not on F1 (0.560 vs 0.577) or recall (0.408 vs 0.521). I chose it anyway because, for a warning app, a wrong "you're fine" is the costly mistake, while an unnecessary warning only costs a short walk. The demo works end to end: it gives a live prediction advice on how much extra walking would raise the chance. The calorie regression was not built, and I dropped it to focus on the classifier and the demo.

Limitations:

- The data is 30 self-selected users from 2016, so it may not represent other people.
- The same users appear in train and test, so scores may be optimistic for a brand-new user.
- The model uses only two features, which caps its accuracy, and early-morning predictions are close to guessing.
- It only knows 8:00am to 9:30pm and a fixed 10k goal, and the advice assumes about 100 steps a minute and doesn't know the user's timetable.
- I excluded 3 high-mismatch users and 42 zero-step days, which reduces the data further.

11. Future Improvements

- **Flexible goal:** predict the final daily total and compare it with any goal. The FinalTotal column is already kept for this.
- **Calorie estimate:** steps alone explain little of total burn (resting metabolism isn't in the data), so this needs per-person information. It would be best shown as a "stop now vs finish the plan" difference.
- **More features:** sleep hours and recent pace.
- Per-user personalisation instead of one model for everyone.
- Tuning the 0.5 cutoff to trade false alarms against missed successes.
- **Other ideas I considered:** sleep prediction, activity-pattern clustering, and heart-rate zones.